

KEYLOGGER TESPİT SİSTEMİ: SAVUNMA AMAÇLI KURAL TABANLI PROCESS ANALİZİ

Abdulmutin

Kastamonu Üniversitesi, Bilgisayar Mühendisliği Bölümü, 2. Sınıf
Nesneye Yönelik Programlama Dersi Proje Raporu

Özet- Bu rapor, ASP.NET Core MVC, Entity Framework Core ve PostgreSQL kullanılarak geliştirilen Keylogger Tespit Sistemi projesini açıklamaktadır. Proje, zararlı yazılım üretmeyen ve canlı sistem üzerinde izleme yapmayan savunma amaçlı bir öğrenci uygulamasıdır. Kullanıcı tarafından manuel girilen process/log benzeri bilgiler, veritabanında tutulan detection rule kayıtları ile karşılaştırılır; tetiklenen kurallar risk puanına dönüştürülür ve sonuç düşük, orta veya yüksek risk olarak sınıflandırılır. Uygulama, nesneye yönelik programlama ilkelerini model sınıfları, servis arayüzleri, servis implementasyonları, controller katmanı ve view model yapıları üzerinden göstermektedir. Ayrıca kimlik doğrulama, rol bazlı yetkilendirme, analiz geçmişi, raporlama, detection rule yönetimi ve etik kullanım sayfası gibi tamamlayıcı modüller içerir.

Anahtar Kelimeler- ASP.NET Core MVC, Nesneye Yönelik Programlama, Entity Framework Core, PostgreSQL, Siber Güvenlik, Kural Tabanlı Analiz.

I. GİRİŞ

Siber güvenlik alanında zararlı yazılım davranışlarını anlamak, yalnızca saldırı araçları geliştirmek anlamına gelmez. Eğitim ortamlarında daha güvenli yaklaşım, zararlı davranış göstergelerini temsil eden örnek kayıtların incelenmesi ve bu kayıtlara göre risk değerlendirmesi yapılmasıdır. Bu proje, keylogger benzeri şüpheli davranışları doğrudan üretmeden, yalnızca manuel girilen process özelliklerini analiz eden bir web uygulaması olarak tasarlanmıştır.

Uygulamanın temel amacı, Nesneye Yönelik Programlama dersinde öğrenilen sınıf, nesne, kapsülleme, arayüz, servis katmanı ve katmanlı mimari kavramlarını gerçek bir web projesi üzerinde göstermektir. Proje aynı zamanda etik kullanım sınırlarını açık biçimde belirtir: tuş kaydı alma, klavye hook kullanma, canlı process takip etme veya zararlı kod üretme özelliği bulunmaz.

II. PROBLEM TANIMI VE AMAÇ

Bir process kaydının riskli olup olmadığını değerlendirmek için yayıncı bilgisi, dosya yolu, başlangıçta çalışma durumu, arka planda çalışma, ağ bağlantısı ve klavye erişimi şüphesi gibi göstergeler birlikte incelenmelidir. Bu göstergelerin ayrı ayrı yorumlanması zor ve hataya açık olabilir. Proje bu problemi, merkezi bir kural tabanı ve puanlama mekanizması ile çözmektedir.

Projenin hedefleri aşağıdaki şekilde özetlenebilir:

- Manuel girilen process bilgilerini kayıt altına almak.
- Detection rule kayıtlarına göre otomatik risk puanı üretmek.

- Risk seviyesini düşük, orta veya yüksek olarak sınıflandırmak.
- Analiz detaylarında tetiklenen kuralları ve önerileri göstermek.
- Admin kullanıcılar için rule ve kullanıcı yönetimi sağlamak.
- Etik kullanım sınırlarını kullanıcıya açıkça göstermek.

III. KULLANILAN TEKNOLOJİLER

Proje .NET 8 hedef çatısı ile hazırlanmış bir ASP.NET Core MVC uygulamasıdır. Veri erişimi için Entity Framework Core, veritabanı olarak PostgreSQL ve PostgreSQL bağlantısı için Npgsql sağlayıcısı kullanılmıştır. Kimlik doğrulama tarafında ASP.NET Core Cookie Authentication tercih edilmiştir. Arayüz katmanında Razor Views ve Bootstrap tabanlı sade bir tasarım bulunmaktadır. Veritabanı ortamı Docker Compose ile ayağa kaldırılmaktadır.

Tablo I. Temel Teknolojiler

Teknoloji	Projedeki Görevi
ASP.NET Core MVC	Controller, View ve Model tabanlı web uygulaması yapısı
Entity Framework Core	ORM, migration, ilişki tanımları ve seed data işlemleri
PostgreSQL	Kullanıcı, analiz, kural ve log verilerinin saklanması
Cookie Authentication	Login, logout ve rol bazlı erişim kontrolü
Bootstrap	Responsive ve okunabilir kullanıcı arayüzü
Docker Compose	PostgreSQL servisinin standart şekilde çalıştırılması

IV. SİSTEM MİMARİSİ

Uygulama katmanlı bir MVC mimarisine sahiptir. Controller sınıfları HTTP isteklerini karşılar, servis arayüzleri üzerinden iş kurallarını çağırır ve Razor View dosyalarına uygun model veya view model gönderir. Veri tabanı işlemleri doğrudan controller içinde yapılmaz; bu işlemler servis sınıflarında toplanır. Bu yaklaşım sorumlulukların ayrılmasını sağlar ve kodun okunabilirliğini artırır.

Tablo II. Katmanlar ve Sorumluluklar

Katman	Örnek Dosyalar	Sorumluluk
Models	User, ProcessReport, DetectionRule	Veri varlıkları ve ilişkiler
Data	ApplicationDbContext	DbSet tanımları, ilişkiler, enum dönüşümleri, seed data
Services	AnalysisService, AuthService	İş kuralları ve veri erişim akışı
Controllers	AnalysisController, UsersController	HTTP akışı ve yönlendirme
ViewModels	ProcessReportCreateViewModel	Form ve ekran modelleri
Views	Razor .cshtml dosyaları	Kullanıcı arayüzü

V. VERİTABANI TASARIMI

Veritabanı tasarımı, analiz sürecini izlenebilir ve raporlanabilir hale getiren ilişkisel bir yapı kullanır. User tablosu kullanıcı bilgilerini, Role tablosu yetki rollerini, ProcessReport tablosu manuel girilen process kayıtlarını tutar. Her process kaydı için bir AnalysisResult oluşturulur. Tetiklenen her kural TriggeredRule tablosunda saklanır. Recommendation tablosu risk seviyesine göre öneriler sunar. AuditLog tablosu ise giriş, çıkış, analiz oluşturma ve yönetim işlemlerinin kaydını tutar.

ApplicationDbContext içinde kullanıcı e-postası için unique index tanımlanmıştır. ProcessReport ile AnalysisResult arasında bire bir ilişki kurulmuştur. DetectionRuleType ve RiskLevel enum değerleri veritabanında string olarak saklanır. Seed data ile Admin ve Student rolleri, varsayılan admin kullanıcısı, başlangıç detection rule kayıtları, öneriler ve örnek yüksek riskli analiz kaydı oluşturulur.

VI. NESNEYE YÖNELİK PROGRAMLAMA YAKLAŞIMI

Projede nesneye yönelik programlama ilkeleri doğrudan uygulanmıştır. Entity sınıfları veriyi ve ilişkileri temsil ederken, servis sınıfları davranışı ve iş kurallarını taşır. Bu ayırım, kapsülleme ve sorumlulukların ayrılması açısından önemlidir.

A. Kapsülleme

Risk hesaplama ayrıntıları AnalysisService içinde tutulmuştur. Controller yalnızca servisi çağırır; puan hesaplama, tetiklenen kural oluşturma ve risk seviyesi belirleme ayrıntılarını bilmez.

B. Soyutlama

IAnalysisService, IAuthService, IUserService, IDetectionRuleService ve benzeri arayüzler controller katmanını somut sınıflardan ayırır. Böylece iş mantığı daha düzenli ve değiştirilebilir hale gelir.

C. Çok Biçimlilik ve Bağımlılık Enjeksiyonu

Servisler uygulama başlangıcında AddScoped ile DI container içine eklenmiştir. Controller sınıfları arayüz tiplerini kullanır. Bu yapı, aynı arayüzün farklı implementasyonlarla kullanılabilmesini sağlayan çok biçimlilik yaklaşımını destekler.

D. Enum Kullanımı

DetectionRuleType ve RiskLevel enum sınıfları, sabit davranış ve sınıflandırmaları tip güvenli şekilde temsil eder. Bu sayede string karşılaştırmalarından doğabilecek hatalar azaltılmıştır.

VII. ANALİZ ALGORİTMASI

Analiz işlemi, kullanıcı tarafından girilen process özelliklerinin aktif detection rule kayıtları ile karşılaştırılmasıyla başlar. Her kural belirli bir davranış göstergesini temsil eder. Kural tetiklenirse ilgili risk puanı toplam puana eklenir ve tetiklenme nedeni kaydedilir. Toplam puan 100 ile sınırlandırılır.

Toplam Risk = min(100, tetiklenen kural puanlarının toplamı)

Risk seviyesi 0-30 arası düşük, 31-60 arası orta ve 61-100 arası yüksek olarak sınıflandırılır. Bu eşikler README dosyasında da belirtilmiştir ve servis içindeki CalculateRiskLevel metodu tarafından uygulanır.

Tablo III. Varsayılan Detection Rule Kayıtları

Kural	Davranış Göstergesi	Puan
Bilinmeyen Yayınıcı	Yayınıcı alanı boş, unknown veya bilinmeyen	15
Başlangıçta Çalışma	Sistem açılışında otomatik çalışma	15
Temp/AppData Konumu	Temp veya AppData konumundan çalışma	20
Klavye Erişimi Şüphesi	Manuel işaretlenen klavye erişimi şüphesi	25
Şüpheli Ağ Bağlantısı	Ağ bağlantısı bulunduğu bildirilmesi	15
Sahte Sistem Dosyası Adı	Sistem dosyası gibi görünen ad kullanımı	20
Arka Planda Çalışma	Arka planda çalışma davranışı	10

VIII. UYGULAMA MODÜLLERİ

Login modülü, kullanıcı e-posta ve şifre bilgisini doğrular ve cookie tabanlı oturum açar. Dashboard modülü toplam analiz sayısını, risk seviyelerine göre sayıları ve son analizleri gösterir. Yeni Analiz modülü, process adı, dosya yolu, yayıncı, ağ bağlantısı, başlangıçta çalışma ve benzeri alanları içeren bir form sunar. Analiz Detay modülü tetiklenen kuralları, toplam puanı, risk seviyesini ve önerileri gösterir.

Admin rolüne sahip kullanıcılar Detection Rules Yönetimi sayfasında kuralları ekleyebilir, güncelleyebilir veya aktif/pasif hale getirebilir. Kullanıcı Yönetimi sayfası yeni kullanıcı oluşturma ve kullanıcı aktiflik durumunu değiştirme işlevi sağlar. Raporlar sayfası toplam analizleri, risk dağılımını ve en sık tetiklenen kuralları gösterir. Etik Kullanım sayfası ise projenin ne yaptığı ve ne yapmadığı bilgisini açıkça ayırır.

IX. GÜVENLİK VE ETİK SINIRLAR

Bu proje keylogger yazmaz, tuş kaydı almaz, canlı process izlemez ve zararlı kod üretmez. Sadece eğitim ve savunma amaçlı örnek veri analizi yapar.

Uygulamada yetkilendirme için [Authorize] öznitelikleri kullanılmıştır. Detection rule ve kullanıcı yönetimi gibi kritik alanlar yalnızca Admin rolüne açılmıştır. Form

gönderimlerinde ValidateAntiForgeryToken kullanılarak CSRF riskine karşı temel koruma sağlanmıştır. Şifreler düz metin olarak saklanmaz; proje düzeyinde sabit salt ile SHA-256 hash üretilir. Ayrıca kullanıcı işlemleri AuditLog tablosuna yazılarak izlenebilirlik artırılmıştır.

X. DOĞRULAMA VE ÖRNEK SENARYO

Seed data içinde svhost.exe adlı örnek şüpheli kayıt bulunmaktadır. Dosya yolu AppData altında verilmiş, yayıncı Unknown olarak işaretlenmiş, başlangıçta çalışma, ağ bağlantısı, klavye erişimi şüphesi, arka planda çalışma ve sahte sistem dosyası adı gibi göstergeler aktif hale getirilmiştir. Bu kayıt tüm varsayılan kuralları tetiklediği için toplam puan 100 olarak sınırlandırılır ve risk seviyesi yüksek olur.

Form doğrulama tarafında zorunlu alanlar ve maksimum uzunluk sınırları view model sınıflarında data annotation ile belirtilmiştir. Böylece eksik veya hatalı veri girişleri kullanıcı arayüzünde yakalanır.

XI. SONUÇ

Keylogger Tespit Sistemi, Nesneye Yönelik Programlama dersi kapsamında geliştirilen, savunma odaklı ve etik sınırları belirgin bir web uygulamasıdır. Proje, ASP.NET Core MVC yapısını, servis tabanlı iş mantığını, Entity Framework Core ile ilişkisel veri modellemeyi ve rol bazlı kullanıcı yönetimini bir araya getirir. Kural tabanlı analiz algoritması basit, anlaşılır ve genişletilebilir yapıdadır. Yeni detection rule türleri, farklı risk puanları veya ek raporlama ekranları eklenerek proje ileride geliştirilebilir.

KAYNAKÇA

- [1] Microsoft, "ASP.NET Core MVC documentation," Microsoft Learn.
- [2] Microsoft, "Entity Framework Core documentation," Microsoft Learn.
- [3] PostgreSQL Global Development Group, "PostgreSQL Documentation."
- [4] OWASP Foundation, "Secure Coding Practices and Web Application Security Guidance."
- [5] Keylogger Tespit Sistemi proje kaynak kodları ve README dosyası.